Logic in CS Fall 2025

Practice Sheet 2

S. Krishna

These questions are meant for your practice and will not be discussed in tutorials. Feel free to discuss about these in Piazza.

Propositional Logic

1. Let P, Q, R be propositional variables. Convert the formula

$$\neg(P \lor (\neg Q \land R)) \to (\neg P \land (Q \lor \neg R))$$

to an equisatisfiable Conjunctive Normal Form (CNF) formula using Tseitin encoding.

2. Discuss the best-case and worst-case time complexities of the DPLL algorithm. Provide a representative example for each scenario.

3. Consider the propositional logic formula φ obtained by conjoining (i.e., "and"-ing) the following subformulas:

Subformula Name	Subformula
φ_1	$x_1 \to (x_2 \lor x_3)$
φ_2	$x_2 \to (\neg x_3 \lor \neg x_4)$
φ_3	$x_3 \to (\neg x_2 \lor x_4)$
φ_4	$x_4 \to \neg(x_1 \lor x_2)$
φ_5	$\neg x_4 \to \neg(\neg x_3 \lor x_3)$

   (a) Convert each one of the subformula above (φ_1 through φ_5) into semantically equivalent CNF formula.

   (b) Check the satisfiability of φ by applying the DPLL algorithm to the CNF formula obtained in the previous sub-question. Make sure you clearly show all the steps.

4. (Hard Question) In this question we will view the set of assignments satisfying a set of propositional formulae as a language and examine some properties of such languages. Let P denote a countably infinite set of propositional variables $p_0, p_1, p_2, \ldots$ Let us call these variables *positional variables*. Let Σ be a countable set of formulae over these positional variables.

   Every assignment $\alpha : P \to \{0, 1\}$ to the positional variables can be uniquely associated with an infinite bitstring w, where $w_i = \alpha(p_i)$. The language defined by Σ—denoted by $L(\Sigma)$—is the set of bitstrings w for which the corresponding assignment α, with $\alpha(p_i) = w_i$ for each natural i, satisfies Σ, that is, for each formula $F \in \Sigma$, we have $\alpha \models F$. In this case, we write $\alpha \models \Sigma$. Let us call the languages definable this way *PL-definable languages*.

(a) Show that PL-definable languages are closed under countable intersection, i.e. if $\mathcal{L}$ is a countable set of PL-languages, then $\bigcap_{L \in \mathcal{L}} L$ is also PL-definable.

(b) Show that PL-definable languages are closed under finite union, i.e. if $\mathcal{L}$ is a finite set of PL-languages, then $\bigcup_{L \in \mathcal{L}} L$ is also PL-definable.

(***Hint.*** *Try proving that the union of two PL-definable languages is PL-definable. The general case can then be proven via induction. If $F = \{F_1, F_2, \ldots\}$ and $G = \{G_1, G_2, \ldots\}$ are two countable sets of formulae, then an infinite bitstring $w \in L(F) \cup L(G)$ if and only if either $w \models F_i$ for all i, or $w \models G_j$ for all j.*)

(c) Show that the language L consisting of all infinite bitstrings except $000\ldots$ (the bitstring consisting only of zeroes) is not PL-definable. You may want to prove the following lemma in order to solve this question:

Lemma 1 *For every PL-definable language L and bitstring $x \notin L$ there exists a finite prefix y of x such that for any infinite bitstring w, $yw \notin L$ (here yw refers to the concatenation of y and w).*

(d) Show that PL-definable languages are closed neither under countable union nor under complementation.

Hint. *Try using the result proven in part (c).*

(e) Show that a PL-definable language either contains every bitstring or does not contain uncountably many bitstrings.

Hint. *Try using the lemma proven in part (c).*

5. (Hard Question) Assume we have a countably infinite list of propositional variables $p_1, p_2, \ldots$. For this problem, by "formula", we always mean a finite string representing a syntactically-correct formula. Let Σ be a set of formulae. For any formula φ, we say $\Sigma \models \varphi$ (read as Σ semantically entails φ) if for any assignment α of the propositional variables that makes all the formulae contained in Σ true, α also makes φ true.

Let us call two sets of formulae Σ_1 and Σ_2 *equisemantic* if for every formula φ, we have $\Sigma_1 \models \varphi$ if and only if $\Sigma_2 \models \varphi$. Furthermore, let us call a non-empty set of formulae Σ *irredundant* if no formula $\sigma \in \Sigma$ is semantically entailed by $\Sigma \setminus \{\sigma\}$.

(a) Show that any set of formulae Σ must always be countable. This implies that we can enumerate the elements of Σ. Assume from now on that $\Sigma = \{\sigma_1, \sigma_2, \sigma_3, \ldots\}$.

(b) Suppose we define Σ' as follows:

$$\Sigma' = \{\ \sigma_1,\ (\sigma_1) \to \sigma_2,\ (\sigma_1 \wedge \sigma_2) \to \sigma_3,\ (\sigma_1 \wedge \sigma_2 \wedge \sigma_3) \to \sigma_4,\ \ldots\ \}.$$

Suppose we remove all the tautologies from Σ' and call this reduced set Σ''. Prove that Σ'' is irredundant and equisemantic to Σ. You can proceed as follows:

i. Show that a non-empty satisfiable set Γ with $|\Gamma| \geq 2$ is irredundant if and only if $(\Gamma \setminus \{\gamma\}) \cup \{\neg\gamma\}$ is satisfiable for every $\gamma \in \Gamma$.

ii. Use the above result to show that Σ'' is irredundant and equisemantic to Σ.

6. (Kinda Hard Question) In this question, we will consider "implications" in the same spirit as "equations" between unspecified propositional formulas. You can think of these as "equations" where the unknowns are propositional formulas themselves, and the $=$ symbol has been replaced by $\rightarrow$.

   Let φ_1 and φ_2 be unknown propositional formulas over $x_1, x_2, \ldots, x_n$. Consider the following implications labelled (1a), (1b) through (na), (nb). A solution to this set of simultaneous implications is a pair of specific propositional formulas (φ_1, φ_2) such that all implications are valid, i.e., evaluate to true for all assignments of variables.

$$\begin{array}{llll} (1a) & (x_1 \wedge \varphi_1) \rightarrow \varphi_2 & (1b) & (x_1 \wedge \varphi_2) \rightarrow \varphi_1 \\ (2a) & (x_2 \wedge \varphi_1) \rightarrow \varphi_2 & (2b) & (x_2 \wedge \varphi_2) \rightarrow \varphi_1 \\ \vdots & \vdots & \vdots & \\ (na) & (x_n \wedge \varphi_1) \rightarrow \varphi_2 & (nb) & (x_n \wedge \varphi_2) \rightarrow \varphi_1 \end{array}$$

   (a) Suppose we are told that φ_1 is $\bot$. How many semantically distinct formulas φ_2 exist such that implications (1a) through (nb) are valid?

   (b) Answer the above question, assuming now that φ_1 is $\top$.

   (c) If we do not assume anything about φ_1, how many semantically distinct pairs of formulas (φ_1, φ_2) exist such that all the implications are valid?

   (d) Does there exist a formula φ_1 such that there is exactly one formula φ_2 (modulo semantic equivalence) that can be paired with it to make (φ_1, φ_2) a solution of the above implications?

7. (Hard Question) Each formula in propositional logic is categorized as either an α (conjunctive) or β (disjunctive) type formula. α formulae (resp. β) have sub-formulae commonly denoted as α_1 and α_2 (resp. β_1 and β_2). In simple terms, an α formula is one which can be written as the conjunction of two other formulae α_1 and α_2. Similarly, a β formula can be written as a disjunction of β_1 and β_2. See Table 1 and 2.

Formula	α_1	α_2
$A \wedge B$	A	B
$\neg(A \vee B)$	$\neg A$	$\neg B$
$\neg(A \rightarrow B)$	A	$\neg B$

Table 1: **Conjunctive (α) Formulas**

Formula	β_1	β_2
$A \vee B$	A	B
$A \rightarrow B$	$\neg A$	B
$\neg(A \wedge B)$	$\neg A$	$\neg B$

Table 2: **Disjunctive (β) Formulas**

We will use some definitions before solving questions.

Definition 2 (Consistency Property) *Let $\mathcal{C}$ be a (possibly infinite) collection of sets of formulae. $\mathcal{C}$ is a consistency property if each $S \in \mathcal{C}$ satisfies the following:*

(a) For each propositional variable p, S does not contain both p and $\neg p$.

(b) $\bot \notin S$ and $\neg\top \notin S$.

(c) If $\neg\neg\phi \in S$, then $S \cup \{\phi\} \in \mathcal{C}$.

(d) If $\alpha \in S$, then $S \cup \{\alpha_1, \alpha_2\} \in \mathcal{C}$ for any alpha formula α with sub-formulae α_1, α_2.

(e) *If $\beta \in S$, then $S \cup \{\beta_1\} \in \mathcal{C}$ or $S \cup \{\beta_2\} \in \mathcal{C}$ for any beta formula β with sub-formulae β_1, β_2.*

Notice that $\mathcal{C}$ is itself a set of sets and any $S \in \mathcal{C}$ can be possibly infinite.

Definition 3 *A propositional consistency property is called **subset closed** if it contains, with each member, all subsets of that member.*

(a) Show that every consistency property $\mathcal{C}$ can be extended to a consistency property that is subset closed, i.e, for any consistency property $\mathcal{C}$, there exists a superset $\mathcal{C}^+ \supseteq \mathcal{C}$ such that $\mathcal{C}^+$ is a subset-closed consistency property.

(b) **Definition 4** *A propositional consistency property, $\mathcal{C}$, has finite character if*

$$S \in \mathcal{C} \text{ iff every finite subset of } S \text{ is in } \mathcal{C}.$$

Show that every propositional consistency property of finite character is subset closed.

(c) **Theorem 5 (Model Existence Theorem)** *If $\mathcal{C}$ is a propositional consistency property and $S \in \mathcal{C}$, then S is satisfiable.*

Theorem 6 (Propositional Compactness Theorem) *Let S be a set of propositional formulae. Then S is satisfiable if and only if every finite subset of S is satisfiable.*

Using Model Existence Theorem, prove Propositional Compactness Theorem.

First Order Logic

8. Let $\mathsf{Student}(x)$ be the predicate that x is a student, $\mathsf{Takes}(x, c)$ be the relation that x takes a course c. Write down first order formulae capturing the following:

   (a) Every student takes at least one course.

   (b) Some students took `CS228`.

   (c) No student takes both `CS228` and `CS310`.

   (d) If a student takes `CS228`, then they must also take `CS230`.

9. Consider the following first order logic formula over the signature $\{P, Q, f\}$, where P and Q are binary predicates and f is a unary function.

$$\varphi \equiv \forall x \left(\exists y \left(P(x, z) \rightarrow \exists z \left(Q(x, z) \rightarrow P(y, f(z))\right)\right) \vee Q(f(y), x)\right).$$

   In the above formula, the scope of every quantifier is explicitly indicated by parentheses immediately after the quantifier. Thus, the scope of $\exists z$ in the above formula is the formula $Q(x, z) \rightarrow P(y, f(z))$ and this is indicated by $\exists z \left(Q(x, z) \rightarrow P(y, f(z))\right)$.

   (a) Find the free and bound variables of φ.

   (b) Write a formula in prenex conjunctive normal form that is semantically equivalent to φ.

(c) A student wants to construct a structure M over the vocabulary $\{P, Q, f\}$ and find two assignments α and β of free variables (if any) of φ, such that $M, \alpha \models \varphi$ and $M, \beta \models \neg\varphi$. Indicate whether this is possible. If your answer is in the positive, you must give M, α, β and clearly show your reasoning why $M, \alpha \models \varphi$ and $M, \beta \models \neg\varphi$. If your answer is in the negative, you must give clear reasoning why it is impossible to find M, α, β such that $M, \alpha \models \varphi$ and $M, \beta \models \neg\varphi$.

10. (Hard Question) Recall Question 1 from Tutorial 5, in which we restricted ourselves to first-order logic without equality; that is, we do not use the atomic formula $x = y$ where x, y are terms. Fix a signature τ and τ-structures $\mathcal{A}, \mathcal{B}$ and assignments α, β. Consider a relation $\sim$ on pairs $(\mathcal{A}, \alpha)$, $(\mathcal{B}, \beta)$ that satisfies the following two properties:

(P1) If $(\mathcal{A}, \alpha) \sim (\mathcal{B}, \beta)$ then for every atomic formula F we have $\mathcal{A} \models_\alpha F$ iff $\mathcal{B} \models_\beta F$.

(P2) If $(\mathcal{A}, \alpha) \sim (\mathcal{B}, \beta)$ then for each variable x we have (i) for each $a \in U^{\mathcal{A}}$ there exists $b \in U^{\mathcal{B}}$ such that $(\mathcal{A}, \alpha[x \mapsto a]) \sim (\mathcal{B}, \beta[x \mapsto b])$, and (ii) for all $b \in U^{\mathcal{B}}$ there exists $a \in U^{\mathcal{A}}$ such that $(\mathcal{A}, \alpha[x \mapsto a]) \sim (\mathcal{B}, \beta[x \mapsto b])$.

If $(\mathcal{A}, \alpha) \sim (\mathcal{B}, \beta)$, then we have shown in tutorial that for any formula F built from atomic formulae using the connectives $\neg, \wedge, \exists$, we have $\mathcal{A} \models_\alpha F$ iff $\mathcal{B} \models_\beta F$.

Now, consider a signature τ containing only unary predicate symbols $P_1, \ldots, P_k$. Using the above result we proved in the tutorial, show that any satisfiable τ-formula has a structure with at most 2^k elements in its universe satisfying it.

Automata Theory

11. Let $\Sigma = \{0, 1\}$.

(a) Construct DFAs for each of the following languages. Try to use as few states as you can in your construction. In the following $n_i(w)$ denotes the count of i's in the string w, for $i \in \Sigma$. Similarly, $|w|$ denotes the length of the string w.

(i) $L_1 = \{0^m 1^n \mid m + n \equiv 0 \pmod 2 \ \wedge \ m \equiv 0 \pmod 3\}$. Think of L_1 as the set of all even length strings of 0s followed by 1s, where the count of 0s is a multiple of 3.

(ii) $L_2 = \{0^m 1^n 0^k \mid m, n, k > 0 \ \wedge \ m + n + k \equiv 0 \pmod 2\}$. Think of L_2 as the set of all strings obtained by inserting a block of 1s inside a block of 0s such that the length of the overall string becomes even.

(iii) $L_3 = \{w \in \Sigma^* \mid w = u \cdot v, \ u, v \in \Sigma^*, \ n_0(u) \equiv 0 \pmod 2, \ n_1(v) \equiv 0 \pmod 2\}$. Think of L_3 as the set of all strings that can be cut into two parts and given to two applications, one of which expects an even count of 0s while the other expects an even count of 1s.

(iv) $L_4 = \{w \in \Sigma^* \mid |w| + 2 \cdot n_1(w) \equiv 0 \pmod 3\}$. Think of L_4 as the set of all strings whose length would be a multiple of 3 if we simply repeated each 1 in the string thrice (without caring about the 0s).

(b) Construct NFAs for each of the following languages. You are allowed to re-use DFAs constructed in the previous part of this question as building blocks of your NFAs.

(i) $L_5 = \{w \in \Sigma^* \mid w = u \cdot v \cdot x,\ u, v, x \in \Sigma^*,\ |u| + 2|v| + 3|x| \equiv 0 \pmod 4\}$. You can think of a string being broken into three parts and fed to three applications, such that the first (resp. second and third) application charges Re. 1 (resp. Rs. 2 and Rs. 3) to process each letter in the string. Then L_5 is the set of strings that can be processed by spending a multiple of 4 Rupees.

(ii) $L_6 = \{w \in \Sigma^* \mid w = u \cdot v,\ (u \in L_1 \wedge v \in L_2) \ \vee\ (u \in L_2 \wedge v \in L_1)\}$. Thus, L_6 is the set of all strings that can be broken into two parts, such that the first part belongs to L_1 and the second part belongs to L_2, or vice versa.

(iii) $L_7 = \{w \in \Sigma^* \mid w = u_1 \cdot u_2 \cdot \ldots \cdot u_k,\ k > 0,\ k \equiv 0 \pmod 2,\ u_i \in L_2 \text{ for all } i\}$. Thus, L_7 is the language of all strings that can be broken up into an even number of strings from L_2.

(iv) $L_8 = \{w \in \Sigma^* \mid n_1(w') \equiv 0 \pmod 2\}$, where w' is obtained from w by replacing every alternate letter starting from the second letter by ϵ. Thus if $w = 0101001$, then $w' = 0001$. You can think of the string w as a sequence of bits coming in so fast that a machine can only read every alternate bit. L_8 is then the sequence of strings that can be accepted by such a lossy automaton for L_2.

12. The solution to this problem is specific to your roll number. Let $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, B\}$. We will say a string $w \in \Sigma^*$ is embedded in a string $u \in \Sigma^*$ iff w can be obtained from u by replacing some letters in u with ϵ. Thus, 014 is embedded in 203421049. To see why this is so, notice that 014 can be obtained as $\epsilon 0 \epsilon\epsilon 1 \epsilon 4 \epsilon$. However, 014 is not embedded in 23421049.

    (a) State your roll number as a string in Σ^*. Let us call this string ρ.

    (b) Give a DFA with no more than $|\rho| + 1$ states that accepts the language $L_\rho = \{w \mid w \in \Sigma^*,\ \rho \text{ is embedded in } w\}$.

    Note: You can of course start with an NFA, and then use the subset construction to convert it to a DFA. But this is a long and tedious route, and will not give you much intuition to understand what each state in the resulting DFA represents. So you are strongly advised not to follow this route. There is enough structure in the problem to permit a much simpler construction of a DFA, and you are encouraged to think about it.

13. Let $\Sigma = \{0, 1\}$. For every word $w \in \Sigma^*$, we say $w' \in \Sigma^*$ is a *subword* of w iff w' is obtained by replacing some letters in w with ε. Thus, 011 is a subword of 0010111, but is not a subword of 0001. (This is same as what you saw in last question, but defined differently.) Furthermore, for $w \in \Sigma^*$, define w^R as the string w read in reverse. For example, 011^R is 110.

    Now, let L be a regular language over Σ. Define $\text{Lossy}(L) := \{w' : w' \text{ is a subword of some word in } L\}$. The need for such languages arises in real life: Imagine a channel on some network, on which we are transmitting bit streams. However, the channel is lossy, and some bits are lost in transmission. Therefore, if we transmit a word w, we may receive a subword of w. In particular, if we want to transmit words from a regular language L_1, we'll actually end up receiving subwords of words in L_1.

    Show that if L is regular, so is $\{w^R : w \in \text{Lossy}(L)\}$, by constructing an NFA for this language.

14. Let L be a regular language. Prove that the following languages are regular:

    (a) $\mathsf{init}(L) := \{\, w : wx \in L \text{ for some } x \in \Sigma^* \,\}$

(b) $L_{\frac{1}{2}} = \{x \mid \exists y, |x| = |y|, xy \in L\}$

(c) $\mathsf{CubeRoot}(L) := \{\, w : w^3 \in L \,\}$

(d) $\mathsf{cycle}(L) = \{uv \mid vu \in L\}$ i.e, the set of all cyclic shifts of words accepted by L.

15. Let $L_n = \{x \mid x$ is a binary number that is a multiple of $n\}$. Show that for all $n \geq 1$, L_n is regular.

16. (Hard Question) Prove that every regular language over a unary alphabet (say, $\Sigma = \{0\}$) is a finite union of languages of the form $L_{c,d} = \{0^{c \cdot n+d} | n \geq 0\}$ for constant integers $c, d \geq 0$.

17. (Hard Question) In this question, we consider non-deterministic finite automata (NFA) without ε-transitions, but with a modification of the acceptance condition studied in class. This modification is motivated by the notion of robust acceptance of words by an NFA.

    Formally, let $A = (Q, \Sigma, Q_0, \delta, F)$ be an NFA, where $Q_0, F \subseteq Q$ and $\delta : Q \times \Sigma \to 2^Q$. We define a run of A on a word $w = a_1 a_2 \ldots a_k$, where each $a_i \in \Sigma$, as a sequence of states $(q_0, q_1, \ldots q_k)$ such that $q_0 \in Q_0$ and $q_i \in \delta(q_{i-1}, a_i)$ for all $i \in \{1, \ldots, k\}$.

    If $w = \varepsilon$, a run of A on w is (q_0), where $q_0 \in Q_0$. A run of A on w is said to be accepting, if the last state in the corresponding sequence of states, i.e. q_k in the above notation, is in F.

    We say that a word $w \in \Sigma^*$ is k-robustly accepted by A iff there are at least k distinct accepting runs of A on w. Here, two runs $(q_0, \ldots, q_k)$ and $(q'_0, \ldots, q'_k)$ of A on a word w of length k (≥ 0) are said to be distinct iff there is some i ($0 \leq i \leq k$) such that $q_i \neq q'_i$.

    Note that the usual notion of acceptance by an NFA coincides with 1-robust acceptance.

    For $k \geq 1$, define

    $$L_k(A) = \{\, w \in \Sigma^* \mid w \text{ is } k\text{-robustly accepted by } A \,\}.$$

    (a) Show that for every NFA A and every $k \geq 1$, $L_k(A)$ is regular.

    (b) Give an example of an NFA A with at most 5 states such that $L_{2i}(A)$ is a strict subset of $L_i(A)$, i.e, $L_i(A) \cap (\Sigma^* \setminus L_{2i}(A)) \neq \emptyset$, for all $i \geq 1$.

18. Let L be a non-empty regular language over an alphabet Σ. We denote by

    - $\mathsf{short}(L)$ the length of the shortest word in L.
    - $\mathsf{det}(L)$ the number of states of the smallest DFA A that accepts L.
    - $\mathsf{nd}(L)$ the minimal number of states in any NFA that accepts L.

    Suppose $|\Sigma| = 1$. Which of the statements below are true?

    (a) For every integer $k \geq 1$, there exists regular language L_k such that $\mathsf{short}(L_k) = \mathsf{nd}(L_k)$ but $\mathsf{nd}(L_k) \neq \mathsf{det}(L_k)$.

    (b) For every integer $k \geq 1$, there exists regular language L_k such that $\mathsf{short}(L_k) = \mathsf{nd}(L_k) - 1$ but $\mathsf{nd}(L_k) = \mathsf{det}(L_k)$.

    (c) For every integer $k \geq 1$, there exists regular language L_k such that $\mathsf{short}(L_k) < \mathsf{nd}(L_k)$ but $\mathsf{nd}(L_k) = \mathsf{det}(L_k) + 1$.

19. Consider the sentence $\varphi = \forall x(Q_a(x) \to \exists y[S(x, y) \wedge Q_b(y)])$. Draw the DFA A for which $L(A) = L(\varphi)$ **exhibiting all the steps** as discussed in class.